Họ tên: Phan Doãn Thành Tài Lớp: MSSV: N25DECE005

Thứ ngày tháng năm

2,5 Prob 1. Singly Linked List - In place Reversal & Pointer Tracing

P1: 2.5 Given a singly linked list head -> [10] -> [25] -> [40] -> [15] ->

P2: 2.5 Null

P3: 2.5 Task 1: Write a function / algorithm in pseudocode to reverse the singly linked list, including 3 pointers (prev, curr, next) to complete your function.

P4: 2.5 Task 2: Draw a diagram ~~of~~ step by step pointer trace table showing the positions & value of the 3 pointers (prev, curr, next) after ~~next~~ each ~~int~~ interaction until the list is fully reversed.

10,0 Note: Do Not Use Algorithms Library

Answer:

1,25 Task 1:

- Function ~~Reversed~~ reverseLinkedList (head):

  prev = Null

  curr = head

  next = Null

  while (curr is not Null):

  next = curr.next

  curr.next = prev

Scanned with CamScanner

Thứ ngày tháng năm

prev = curr

curr = next

head = prev

Return head

1,25 Task 2:

Iteration	next	curr.next	prev	curr	Status of curr. linked list
Start	Null	[10]	Null	[10]	10 -> 25 -> 40 -> 15 -> Null
1	[25]	Null	[10]	[25]	Null <- 10, 25 -> 40 -> 15 -> Null
2	[40]	[10]	[25]	[40]	Null <- 10 <- 25, 10 -> 15 -> Null
3	[15]	[25]	[40]	[15]	Null <- 10 <- 25 <- 40, 15 -> Null
4	Null	40	[15]	Null	Null <- 10 <- 25 <- 40 <- 15

2,5 Prob 2: Circular linked list - Insert at head

Given a singly circular linked list: head -> [10] ->

Scanned with CamScanner

[20] -> [30] -> points back to head

1.25 Task 1: write pseudo code for the function InsertHead (head, new value) to insert a new node containing new value at the front of the circular linked list while maintaining its circular structure

1.25 Task 2: Trace the execution steps (via text description & pointer diagrams) when inserting new value = 5 into given list

Answer:

Task 1: Function InsertHead (head, newValue):

1. newNode = CreateNode (newValue)
   If (head == Null)
      newNode.next = newNode
      head = newNode
      Return head
2. tail = head
   while (tail.next != head)
      tail = tail.next
3. newNode.next = head
   tail.next = newNode
   head = newNode

Scanned with CamScanner

Thứ ngày tháng năm

Return head

Task 2:

Step 1: Create newNode have 5. newNode -> [5]

Step 2 (tail Racing):

Start from tail = head ([10])

Scan the list until tail.next == head

Current position is [30] bcz [30].next

Returning head ([10]). So tail = [30]

Step 3: Give ~~new Node = tail.next = new Node~~

newNode.next = head -> [5] # Return to [10]

Give tail.next = newNode -> [30] move to [5] instead [10]

Step 4: Give head = newNode -> pointer move to [5]

2.5 Prob 3: Doubly Linked list - Insert Before Target & Edge Cases

~~2.75~~ Task 1: Write pseudo code for InsertBefore (head, target, newValue) to insert a newNode containing newValue immediately before the first occurrence of target

Scanned with CamScanner

(2)

Task 2: List the exact order of ~~pointers~~ pointer assignments required & briefly justify why this specific sequence avoids memory leaks or broken links

Task 3: Explicitly handle the following 2 edge cases in pseudocode / code

1. keyget is located at the head of the node
2. keyget does not exist in the list

Answer:

1.5 Task 1 & 3:

```
Function InsertBefore(head, keyget, newValue)
    If head is Null:
        Print "Error list is empty"
        Return head
    newNode = Create Node(newValue)
Edge case 1: ~~keyget~~
    If head.value == keyget:
        newNode.next = head
        head.prev = newNode
        head = newNode
        Return head
```


Scanned with CamScanner

Thứ ngày tháng năm

curr = head

While curr is not Null and curr.value != curr.target

curr = curr.next

Edge Case 2:

If curr is Null:

print ("Error: target not found")

Delete Node (new Node)

return head

new Node.next = curr

new Node.prev = curr.prev

curr.prev.next = new Node

curr.prev = new Node

Return head

0,75 Task 2

Step 1+2: Create new Node in surrounding environment without changing other pointers in the list. new Node.prev = curr.prev makes new Node stores the link of other prev curr.

Step 3: Change the link of prev node. Update the prev node next instead of curr. We copy

Scanned with CamScanner

this link into newNode.prev

Step 4: If step 4 is proceeded before step 3 then we'll lose pointer to curr.prev; then we can not find the link of next. So that it created broken list and memory leak

2.5 Prob 4: Sorted linked list - In order Insertion

Task 1: Write the function SortedInsert(head, val) to insert val into the list such that the ~~sorting~~ ascending order is preserved

Task 2: Construct a step by step trace table detailing pointer ~~the~~ the algorithm & condition evaluations when inserting val = 6 into the list

Answer:

1.25 Task 1:

```
Function SortedInsert(head, val)
    newNode = CreateNode(val)
    newNode.next = Null
Case 1:  If head is Null or val <= head.val
        newNode.next = head
        head = newNode
        Return head
```


Scanned with CamScanner

Thứ ngày tháng năm

Case 2 while (curr.next is not Null & curr.next.value < val):
curr = curr.next
newNode.next = curr.next
curr.next = newNode
Return head.

1.25 Task 2:

Step	Position	Condition in while()	Res	Next	Status
Start	[2]	-	-	Start loop	h->2->5->8->Null
1	[2]	curr.next != Null and curr.next.val<6 (5<6)	True	curr = curr->curr.next [2]to curr->[5]	
2	[5]	curr.next != Null and curr.next.val<6 (8<6)	False	End loop	Curr stop at [5]

Scanned with CamScanner